BH Project Implementation Plan

Generated for the Bravehearts Basketball Club project.

Recommended execution order

- 1. Phase 0 — Baseline and safety checkpoint
- 2. Phase 1 — Repository hygiene and README
- 3. Phase 2 — Configuration consistency and health endpoints
- 4. Phase 3A — SQLite-first production cleanup
- 5. Phase 4 — Route correctness and API alignment
- 6. Phase 5.1 — Redact sensitive logs
- 7. Phase 5.2 — Password hashing upgrade
- 8. Phase 5.3 — Admin session expiration
- 9. Phase 6 — Standard errors and input validation
- 10. Phase 7 — Pagination
- 11. Phase 8 — Transactions and ID generation
- 12. Phase 9 — Upload/image hardening
- 13. Phase 10 — Lint, CI, Docker
- 14. Phase 11 — Observability and backups

- 15. Phase 12 — Larger refactors/future scaling

Phase 0 — Baseline and safety checkpoint

Goal

Create a clean starting point before changing behavior.

Tasks

- Confirm Git state: `git status`
- Create a working branch: `git checkout -b harden-production-readiness`
- Run baseline checks: `npm test -- --run` and `npm run build`
- Record current known behavior: API routes, admin login flow, database file location, deployment expectation.

Acceptance criteria

- Current tests pass.
- Current build passes or known build errors are documented.
- No implementation starts before the baseline is understood.

Phase 1 — Repository hygiene and secret safety

1.1 Protect secrets and runtime files

Goal

Ensure generated files, local data, logs, uploads, and secrets are not committed.

Tasks

- Review tracked files: `git ls-files`
- If tracked, untrack runtime/generated files without deleting local copies:

```
- .env
- node_modules/
- dist/
- logs/
- uploads/
- data/database.sqlite
```

- Update `.gitignore` for environment files, dependencies, build output, logs, uploads, SQLite databases, coverage, and OS/editor files.
- Verify `.env.example` contains no real secrets.

Acceptance criteria

- `.env` is not tracked.
- SQLite database is not tracked.
- logs, uploads, `dist`, and `node_modules` are not tracked.
- `.env.example` remains tracked.

1.2 Add project documentation

Goal

Make the project easy to run and maintain.

Tasks

Create `README.md` with:

- project overview
- tech stack
- local setup
- environment variables
- database setup
- migrations
- tests
- build
- deployment notes
- backup/restore process
- admin access/bootstrap instructions

Acceptance criteria

- A new developer can run the app locally using the README.
- Required environment variables are documented.

Phase 2 — Configuration consistency

2.1 Fix frontend API base URLs

Goal

Remove hardcoded localhost from frontend code.

Tasks

- Replace public API base with `import.meta.env.VITE_API_BASE_URL || '/api'`.
- Replace admin API base with `import.meta.env.VITE_ADMIN_API_BASE_URL || '/api/admin'`.
- Add these values to `.env.example`.
- Test local development behavior.

Acceptance criteria

- Production build no longer calls localhost.
- API still works in local development.
- Build passes.

2.2 Add health and readiness endpoints

Goal

Provide stable operational endpoints instead of using `/api/updates` as a healthcheck.

Tasks

- Add `GET /healthz`.
- Add `GET /readyz`.
- `/healthz` returns server liveness.
- `/readyz` checks database connectivity.
- Update Docker healthcheck to call one of these endpoints.

Acceptance criteria

- `GET /healthz` returns 200.
- `GET /readyz` returns 200 only when DB is reachable.
- Docker healthcheck uses the new endpoint.

Phase 3 — Database strategy decision and cleanup

Decision point

Choose one of these paths before implementation.

Option A — SQLite-first production

Best if the project is hosted on one small VPS/server and does not need horizontal scaling soon.

Tasks

- Make `server/db.js` respect `DB_PATH=./data/database.sqlite`.
- Remove PostgreSQL service from `docker-compose.yml`, or clearly mark it unused.
- Update CI to stop starting PostgreSQL unless it is actually used.
- Keep backups focused on SQLite.
- Document SQLite production backup/restore in README.
- Ensure migrations or schema creation are consistent.

Acceptance criteria

- App uses the configured `DB_PATH`.
- CI reflects SQLite usage.
- Docker does not imply PostgreSQL is active when it is not.
- Backup/restore documentation is clear.

Option B — PostgreSQL/Knex production

Best if the project should become a scalable production system.

Tasks

- Introduce a DB access layer using Knex.
- Convert routes gradually from raw sqlite3 callbacks to Knex.
- Use migrations as the only schema source.
- Remove automatic table creation from `server/db.js`.
- Ensure production uses PostgreSQL.
- Ensure tests run against either SQLite memory through Knex or test PostgreSQL in CI.
- Add migration execution to deploy flow.

Acceptance criteria

- Runtime database matches environment.
- CI tests the same DB strategy or an intentionally documented equivalent.
- No duplicate schema definitions exist.

Recommended path

Use Phase 3A first: SQLite-first production cleanup. It is lower risk, faster, and matches the current codebase. A full PostgreSQL migration can become a later dedicated project.

Phase 4 — Fix route correctness and API consistency

4.1 Fix Express route ordering issues

Goal

Prevent dynamic routes like `/:id` from capturing admin routes.

Tasks

- Review route files for dynamic routes before static/admin routes.
- Move static/admin routes above dynamic routes.
- Add regression tests for admin routes.

Acceptance criteria

- Admin routes are no longer intercepted by public `/:id`.
- Tests cover the fixed route ordering.

4.2 Align frontend/admin route expectations with backend

Goal

Ensure frontend API clients match backend routes.

Tasks

- Compare `src/admin-api.js` expected endpoints with backend routes.
- Decide canonical admin route style, recommended: `/api/admin/products`, `/api/admin/games`, `/api/admin/news`, `/api/admin/gallery`, `/api/admin/standings`.
- Keep backward-compatible aliases temporarily if needed.
- Remove duplicates after frontend is migrated.

Acceptance criteria

- Admin frontend calls existing backend endpoints.
- Tests cover key admin endpoints.
- Deprecated routes are documented.

4.3 Start API versioning

Goal

Prepare the API for long-term stability.

Tasks

- Add `/api/v1/*` route aliases while keeping `/api/*` working.
- Update frontend to use `/api/v1` if desired.
- Document legacy `/api` as backwards-compatible.

Acceptance criteria

- `/api/v1/products`, `/api/v1/games`, etc. work.
- Existing `/api/products`, `/api/games` still work.
- No breaking change to current frontend.

Phase 5 — Security hardening

5.1 Stop logging sensitive request bodies

Goal

Prevent passwords, tokens, and payment details from being written to logs.

Tasks

- Update error logger sanitization.
- Redact fields such as password, newPassword, token, authorization, mobileMoneyNumber, sessionId, x-session-id, SMTP password, reset token.
- Avoid logging full request bodies by default in production.

Acceptance criteria

- Sensitive fields are redacted in logs.
- Tests or manual checks confirm no password/token is logged.

5.2 Upgrade password hashing

Goal

Replace custom deterministic hashing with bcrypt or argon2.

Recommended choice

Use bcrypt.

Tasks

- Install bcrypt.
- Update registration to hash with bcrypt.
- Update login to compare with bcrypt.
- Support legacy password hashes temporarily if existing users/admins exist.
- On successful login with legacy hash, verify old hash, rehash with bcrypt, and update database.
- Update password reset to store bcrypt hashes.
- Update admin login if separate.

Acceptance criteria

- New passwords use bcrypt.
- Existing users can still log in if legacy hashes exist.
- Legacy hashes are upgraded on successful login.
- Auth tests pass.

5.3 Add admin session expiration

Goal

Reduce risk from stolen or old admin session IDs.

Tasks

- Add `expires_at` and `last_seen_at` to `admin_sessions`.
- Update login to set expiry.
- Update auth middleware/session loader to reject expired sessions.
- Update logout to delete sessions.
- Optionally rotate session ID on login.

Acceptance criteria

- Expired sessions cannot access admin APIs.
- Logout invalidates session.
- Tests cover valid, missing, expired, and deleted sessions.

5.4 Improve admin authentication transport

Goal

Move toward safer browser authentication.

Recommended target

Use secure HTTP-only cookies for admin sessions.

Tasks

- Add cookie-based session support.
- Keep `x-session-id` temporarily for backward compatibility.
- Set cookies with httpOnly, secure in production, and sameSite lax.
- Add CSRF protection for cookie-authenticated state-changing requests.
- Update admin frontend.

Acceptance criteria

- Admin session can be stored in an HTTP-only cookie.
- Header-based session still works during transition.
- CSRF protection exists for cookie-authenticated mutations.

5.5 Tighten CORS and CSP

Goal

Reduce browser attack surface.

Tasks

- Replace single `CORS_ORIGIN` with list-based `CORS_ORIGINS`.
- Reject unknown origins in production.
- Remove or reduce `unsafe-inline` from CSP where possible.

- Make `connectSrc` environment-aware.

Acceptance criteria

- Only configured origins can call the API in browser context.
- CSP works in production without blocking required assets.
- Local development remains convenient.

Phase 6 — Validation and error response consistency

6.1 Standardize error response format

Goal

Make API errors predictable.

Target format

```
json
{
  "error": {
    "code": 400,
    "message": "Validation failed",
    "status": "INVALID_ARGUMENT",
    "details": []
  }
}
```

Tasks

- Create helper functions for sending errors.

- Update global error handler.
- Update validation middleware.
- Gradually update routes that return raw errors.
- Ensure frontend `handleResponse` supports both old and new format during transition.

Acceptance criteria

- New errors use consistent format.
- Frontend can display errors.
- No stack traces in production responses.
- Tests updated.

6.2 Add query and params validation

Goal

Validate all inputs, not only request bodies.

Tasks

Add Joi schemas for path params, pagination query, search query, order status, game status, poll vote params, notification user IDs, and stats IDs.

Acceptance criteria

- Invalid params return 400.

- Invalid query values return 400.
- Tests cover invalid inputs.

Phase 7 — Pagination and API scalability

7.1 Add pagination to public list endpoints

Goal

Avoid unbounded database reads.

Start with

- `GET /api/news`
- `GET /api/gallery`
- `GET /api/games`
- `GET /api/products`
- `GET /api/players`
- `GET /api/standings`

Recommended transition

Support old behavior temporarily, but default to a reasonable limit. Use `pageSize` and `pageToken` where practical.

Acceptance criteria

- List endpoints limit returned rows.
- `pageSize` has a maximum.
- Frontend still works.
- Tests cover pagination.

7.2 Add pagination to admin list endpoints

Goal

Prevent slow admin pages as data grows.

Start with

- admin orders
- admin users
- admin logs
- admin notifications

Acceptance criteria

- Admin list endpoints support pagination.
- Frontend supports loading more or paged navigation.

Phase 8 — Data integrity and transactions

8.1 Add transactions for multi-step writes

Goal

Prevent partial data writes.

Candidate flows

- order creation
- payment confirmation
- password reset token creation
- poll voting
- admin create/update with audit logs
- player stats updates

Tasks

- Create transaction helper for SQLite or Knex.
- Wrap multi-step operations.
- Add tests for failure rollback where practical.

Acceptance criteria

- Multi-table writes are atomic.

- Failed second step does not leave partial data.

8.2 Improve ID generation

Goal

Avoid collisions.

Tasks

Replace random product IDs with `crypto.randomUUID()` or a prefixed UUID.

Acceptance criteria

- Product ID collisions are practically impossible.
- Tests updated if they assume old ID format.

Phase 9 — File upload and asset hardening

9.1 Harden upload validation

Goal

Prevent unsafe file uploads.

Tasks

- Enforce MIME allowlist: image/jpeg, image/png, image/webp.
- Limit upload size.

- Reject unknown extensions.
- Generate safe filenames using UUID.
- Store uploads outside source-controlled paths where possible.
- Add tests for invalid upload types.

Acceptance criteria

- Unsafe file types are rejected.
- Large files are rejected.
- Filenames are not user-controlled.

9.2 Optimize gallery/static images

Goal

Improve performance.

Tasks

- Compress large images.
- Generate WebP versions.
- Use lazy loading.
- Avoid bundling unnecessary images into initial JS.

- Consider serving gallery from database/uploads instead of bundling all assets.

Acceptance criteria

- Smaller build/static payload.
- Gallery still loads correctly.
- Lighthouse/performance improves.

Phase 10 — Developer experience and CI/CD

10.1 Add linting and formatting

Goal

Prevent style drift and common bugs.

Tasks

- Install ESLint and Prettier.
- Add scripts: lint, format, format:check.
- Add config files.
- Run and fix issues.

Acceptance criteria

- `npm run lint` passes.
- `npm run format:check` passes.
- CI runs both.

10.2 Modernize GitHub Actions

Goal

Keep CI secure and current.

Tasks

- Upgrade GitHub Actions versions.
- Align CI database with selected DB strategy.
- Run lint, tests, and build in CI.

Acceptance criteria

- CI passes.
- CI does not start unused services.
- CI matches production assumptions as much as possible.

10.3 Improve Docker production build

Goal

Build reliable production images.

Tasks

- Convert Dockerfile to multi-stage build.
- Run frontend build inside Docker image build.
- Copy only necessary runtime files.
- Do not copy `.env`, local DB, logs, uploads, or `node_modules`.
- Add or fix `nginx.conf`, or remove nginx service if unused.

Acceptance criteria

- Docker image builds from clean checkout.
- Container starts without requiring committed `dist`.
- Healthcheck passes.
- No secrets copied into image.

Phase 11 — Observability and operations

11.1 Improve logging

Goal

Make logs useful and safe.

Tasks

- Add request IDs.
- Sanitize sensitive fields.
- Log structured errors.
- Avoid excessive logs for successful requests in production if not needed.
- Add log rotation or document external log management.

Acceptance criteria

- Errors can be traced by request ID.
- Logs do not expose secrets.
- Log files do not grow indefinitely without a plan.

11.2 Add backup and restore verification

Goal

Ensure data can be recovered.

Tasks

- Review `scripts/backup-database.js`.
- Add restore instructions.
- Test backup and restore locally.

- Add scheduled backup recommendation for production.

Acceptance criteria

- Backup can be created.
- Restore process is documented and tested.
- Production operator knows where backups are stored.

Phase 12 — Larger refactors and future improvements

12.1 Introduce repository/service layer

Goal

Reduce duplicated SQL and make the app easier to test.

Tasks

Create modules such as:

- `server/repositories/users.js`
- `server/repositories/games.js`
- `server/repositories/products.js`
- `server/services/orders.js`
- `server/services/auth.js`

Acceptance criteria

- Routes become thin.
- Business logic moves to services.
- Database queries are centralized.

12.2 Move fully to Knex/PostgreSQL if needed

Goal

Scale beyond single-server SQLite.

Tasks

- Convert DB layer to Knex.
- Use PostgreSQL in production.
- Migrate existing SQLite data.
- Update Docker, CI, and deploy.
- Add migration and rollback procedures.

Acceptance criteria

- PostgreSQL is the production DB.

- Migration preserves existing data.
- CI tests against PostgreSQL.

12.3 Add OpenAPI documentation

Goal

Document API contract.

Tasks

- Add OpenAPI spec.
- Document resources, methods, request/response bodies, and errors.
- Serve docs locally or generate static docs.

Acceptance criteria

- API consumers can understand available endpoints.
- Error formats and pagination are documented.

First implementation batch

Start with this low-risk batch:

- 1. Create `README.md`.

- 2. Update `.gitignore`.
- 3. Replace frontend hardcoded API URLs.
- 4. Add `/healthz` and `/readyz`.
- 5. Update Docker healthcheck.
- 6. Run `npm test -- --run`.
- 7. Run `npm run build`.

This batch improves deployment readiness immediately and does not require major database or auth migrations yet.